SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

*I **Lokesh S (192511037)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:

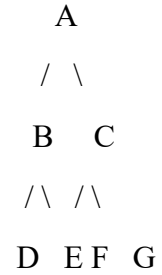
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1. Breadth-First Search (BFS)

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def display_graph():
    print("\nGraph Representation")
    for node, neighbors in graph.items():
        print(f"{node} -> {' '.join(neighbors) if neighbors else 'None'}")

def bfs(start):
    visited = set([start])
    queue = deque([start])

    print("\nBFS Traversal:", end=" ")

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

display_graph()
bfs("A")
```

Output:

```
Graph Representation
A -> B C
B -> D E
C -> F G
D -> None
E -> None
F -> None
G -> None

BFS Traversal: A B C D E F G
```

2. Depth First Search (DFS):

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}


def display_graph():
    print("\nGraph Representation")
    for node, neighbors in graph.items():
        print(f"{node} -> {' '.join(neighbors) if neighbors else 'None'}")


def dfs(node, visited):
    visited.add(node)
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(neighbor, visited)


display_graph()

print("\nDFS Traversal:", end=" ")

visited = set()
dfs('A', visited)

print()
```

Output:

```
Graph Representation
A -> B C
B -> D E
C -> F G
D -> None
E -> None
F -> None
G -> None

DFS Traversal: A B D E C F G
```

3.Eight Puzzle Solver:

```
from collections import deque

GOAL = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

START = ((1, 2, 3),
         (4, 0, 6),
         (7, 5, 8))


def print_state(state):
    for row in state:
        print(" ".join(str(x) if x != 0 else "_" for x in row))
    print()


def get_neighbors(state):

    state = [list(row) for row in state]

    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    directions = [(-1,0),(1,0),(0,-1),(0,1)]

    neighbors = []

    for dx, dy in directions:

        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:

            new_state = [row[:] for row in state]

            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            neighbors.append(tuple(tuple(r) for r in new_state))

    return neighbors


def bfs(start, goal):
```

```
    queue = deque([(start, [])])

    visited = {start}

    while queue:

        current, path = queue.popleft()

        if current == goal:

            print("\nGoal State Reached!\n")

            for state in path + [current]:
                print_state(state)

            return

        for neighbor in get_neighbors(current):

            if neighbor not in visited:

                visited.add(neighbor)

                queue.append((neighbor, path + [current]))

    print("No Solution Found")


print("Initial State:\n")
print_state(START)

print("Goal State:\n")
print_state(GOAL)

print("Searching...\n")

bfs(START, GOAL)
```

Output:

```
Initial State:

1 2 3
4 _ 6
7 5 8

Goal State:

1 2 3
4 5 6
7 8 _

Searching...


Goal State Reached!

1 2 3
4 _ 6
7 5 8

1 2 3
4 5 6
7 _ 8

1 2 3
4 5 6
7 8 _
```